

Robot Walk Recognizer

Did the robot make it back home?

One walk over $\{N, S, E, W\}$ · three machines · three answers

Pramish Sihali · Rista Shrestha · Raksha Shrestha · Manish Yadav

TECH 315: Models of Computation · College of Technology and Engineering

Westcliff University · Instructor: Prof. Kunwar

GOALS

What we set out to build

A **language-recognition system**: hand it a string, it answers **accept or reject**.

One that integrates **all three models** from the course — a finite automaton, a context-free grammar with its pushdown automaton, and a Turing machine.

One problem, one alphabet, three escalating questions — so the three models can be compared directly instead of being three unrelated demos.

The rule we set ourselves: **the audience must be able to check every answer by eye**, with no specialist knowledge.

That last rule is what ruled out an arithmetic-expression validator (it reads as a calculator) and a music-notation checker (it needs music literacy).

THE PROBLEM

One walk, written down

Input is a single walk over the alphabet $\Sigma = \{\mathbf{N}, \mathbf{S}, \mathbf{E}, \mathbf{W}\}$. One symbol is one unit step — north, south, east or west.

NNESSW = **north, north, east, south, south, west**

* *
* *
H *

H = home
* = passed through

Every question we ask is a **yes/no question about this string**.

That is exactly what a language-recognition problem is: is this string in the language, or not?

And because the walk can be drawn, **anyone in this room can check our answers**.

Three questions, three machines

Tier	Machine	The question	Language	In plain terms
1	DFA M1	Did it ever immediately undo a move?	no NS, SN, EW or WE	a goldfish — it remembers only the last move
2	PDA P2 + CFG G2	Did it end on its starting row?	$\#N = \#S$	a stack of sticky notes — one note per north step
3	TM M3	Did it end exactly back home?	$\#N = \#S$ and $\#E = \#W$	a notebook — crossing off pairs on a paper strip

The same input string goes to **all three machines**. No tier is allowed to short-circuit another — the point is to **see all three answers**.

Each question provably needs **strictly more machinery** than the one above it: finite memory, then one counter, then two.

Tier 1 — the finite automaton

$M1 = (Q1, \Sigma, \delta1, q0, F1)$ with $Q1 = \{q0, qN, qS, qE, qW, qdead\}$ and $F1 = Q1 \setminus \{qdead\}$.

The state **is** the last move made — that is the only thing this machine remembers.

Read **S** while in state **qN** and you have undone your last step: go to **qdead** and never come back. Any symbol outside Σ does the same.

$\delta1$ is 24 explicit entries. The implementation looks up (state, symbol) in a table. It never counts anything.

NNESSW	$q0 \rightarrow qN \rightarrow qN \rightarrow qE \rightarrow qS \rightarrow qS \rightarrow qW$	ACCEPT
NS	$q0 \rightarrow qN \rightarrow qdead$	REJECT

Tier 2 — the pushdown automaton and the grammar

$P2 = (\{q_{start}, q_{loop}, q_{accept}\}, \Sigma, \{N, S, \$\}, \delta2, q_{start}, \{q_{accept}\})$ — deterministic, 10 transitions.

A signed tally, not a nesting check. $\#N = \#S$ does not care about order, so the stack pushes on a fresh direction and pops when the opposite one cancels it. Depth is always $|\#N - \#S|$. E and W leave the stack untouched.

G2: $B \rightarrow NBSB \mid SBNB \mid EB \mid WB \mid \epsilon$ — one variable, because “balanced” is the only property being tracked.

Accepted walks get a **printed leftmost derivation**, so the grammar and the machine can be seen to agree.

$B \Rightarrow NBSB \Rightarrow NNBSBSB \Rightarrow NNEBSBSB \Rightarrow NNEBSBSB \Rightarrow NNESSB \Rightarrow NNESSWB \Rightarrow NNESSW$

Why a stack is not optional

Claim. $L2 = \{ w : \#N(w) = \#S(w) \}$ is **not regular** — so no DFA, however many states, can answer question 2.

Proof sketch. Regular languages are closed under intersection with a regular language. Intersect $L2$ with N^*S^* and you get $\{ N^nS^n \}$.

Apply the pumping lemma (Sipser, Thm 1.70) to $s = N^pS^p$: any pumped piece y lies inside the block of N 's, so xy^2z has more N 's than S 's and leaves the language.

So $\{ N^nS^n \}$ is not regular, therefore **$L2$ is not regular.**

In plain terms: a goldfish cannot count. You need the sticky notes.

Tier 3 — the Turing machine

Tape alphabet $\{N, S, E, W, \times, \sqcup\}$. The input is written on the tape and a head moves one cell at a time.

PAIR-CHECK(X, Y): scan right for an unmarked X and cross it off, scan on for a Y and cross that off, return to the left end, repeat. Leftovers of either kind $\rightarrow$ reject.

Run it **twice** — once as (N, S), then as (E, W). **16 states, 84 transitions**, built by instantiating one subroutine twice.

Everything pairs up $\rightarrow$ **accept. The machine always halts: it is a decider.**

$NNESSW \rightarrow xNESSW \rightarrow xNExSW \rightarrow xxExSW \rightarrow xxExxW \rightarrow xxxxxW \rightarrow xxxxxx$
 phase 1: N against S phase 2: E against W

Why a tape is not optional

Claim. $L3 = \{ w : \#N = \#S \text{ and } \#E = \#W \}$ is **not context-free** — so no PDA can answer question 3.

Proof sketch. Context-free languages **are** closed under intersection with a regular language. Intersect $L3$ with $N^*E^*S^*W^*$ and you get $\{ N^nE^mS^nW^m \}$.

The CFL pumping lemma (Sipser, Thm 2.34) rules that out: one stack cannot keep two independent counts that interleave. So **$L3$ is not context-free**.

The bonus result. $\#N = \#S$ is context-free. $\#E = \#W$ is context-free. Their intersection is not — a live demonstration that **CFLs are not closed under intersection**.

Building it

Python 3.9+, standard library only. 9 files, 1,181 lines. One package, one command-line driver.

The rule we held ourselves to: every tier is a real formal machine, not a counter wearing a machine's name.

Tier 1 reads its next state out of a dictionary keyed by (state, symbol). It never counts.

Tier 2 owns a Stack object exposing only push, pop, top and is_empty — the transition table decides which to call.

Tier 3 owns a Tape object with a head that moves exactly one cell per step.

Pipeline: one input → DFA → PDA → TM, each printing its own trace. The three tiers share nothing but the input string.

Testing it

161 tests, all passing, about ten seconds. Plain unittest, no third-party framework.

The oracle principle. A separate, deliberately naive answerer — a regular expression for reversals, `str.count` for the balances — decides what the right answer is. **The machines are never graded by their own logic.**

Exhaustive to length 8: all **87,381** walks through all three machines. Plus 5,000 seeded random walks, 248 constructed near-misses, and a single 100,000-symbol input.

6 of the 8 possible verdict combinations actually occur. The two that never appear are exactly the ones $L3 \subseteq L2$ forbids — the test census confirms the theory.

What it costs

Measured in **machine steps**, not seconds — seconds would measure our laptop.

Tier 1: exactly n steps, 6 states, always. **Tier 2: $n + 2$ steps**, stack depth $|\#N - \#S|$.

Tier 3 takes exactly $(n+1)(n+10)$ steps on every accepted walk of length n — not an upper bound, the exact count, independent of how the letters are arranged.

walk	n	steps measured	$(n+1)(n+10)$
NS	2	36	36
NESW	4	70	70
NNESSW	6	112	112
NNEESSWW	8	162	162

Verified against **all 1,398,101** accepted walks of length ≤ 10 .

The worst case is an accepted input — rejections quit early. NES is rejected in 40 steps, not the 52 the formula would give.

What went wrong, and what we did

Finding a problem the room can check. We rejected an arithmetic validator (reads as a calculator) and a music-notation checker (needs music literacy) before landing on a walk anyone can trace with a finger.

Keeping the machines honest. The easy version of this project is three counters with impressive names. We wrote the transition tables, the stack and the tape instead — and accepted that it would be slower.

A documentation error. Weeks 4 and 5 claimed NEWS was accepted by Tier 1 despite doubling back. It contains **EW**, which is a reversal, so it is **rejected**. Exhaustive testing in Week 6 caught what proofreading had not.

A real defect. The Turing machine had a fixed step limit, so any accepted walk longer than 1,408 symbols **raised an error instead of deciding** — it was not a decider at all above that size. Found in Week 7 by **measuring**, not testing. Fixed, with a regression test.

Where the model stops

Unit steps are a unary encoding of distance. NNN means “north 3”. Allow decimal distances — N3 E2 — and the same balance check provably **exceeds what a PDA can do.**

The input encoding, not the question, decides which machine you need. That is the single most useful thing we learned.

Real motion is continuous. 2.37 metres, arbitrary headings, GPS noise. Sensors must discretise the world into a string first, so the answer is **exact about the string and approximate about the world.**

Our Turing machine always halts — it is a decider. The halting problem shows that not every Turing machine does. Our problem sits safely below that ceiling.

Live demonstration

Look at the picture and predict the three answers before the machines do.

What the project shows

One string, three questions, three machines — and each machine is **provably the weakest one that can answer its question**.

The hierarchy is not an abstraction here: you can **watch** finite memory run out, then **watch** one counter run out.

Everything is checked — **161 tests**, an independent oracle, and a measured cost that turned out to be **exact**.

Questions?

Sipser, M. (2013). Introduction to the theory of computation (3rd ed.). Cengage Learning.